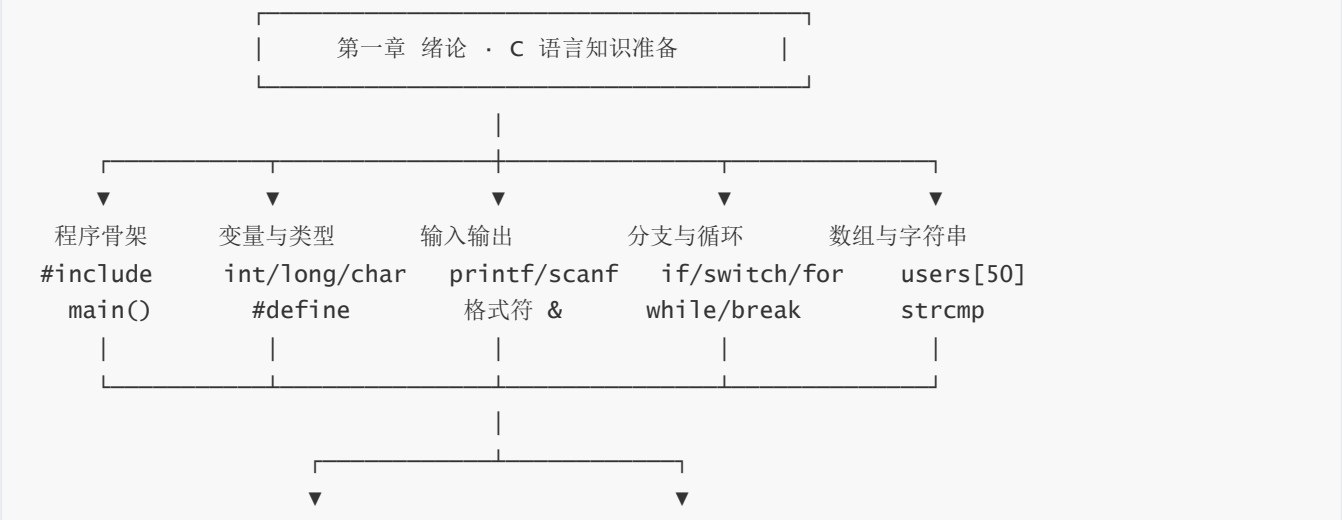
第一章 数据结构绪论 · C语言知识准备

> 配套：《第一章 第一讲数据结构绪论-理论部分》《第一章 第二讲数据结构绪论-实践部分》
> 定位：绪论这一章要能**读懂并改写登录模块的代码**，为此需要掌握的 C 语言基础语法，全部在这里补齐。
> 分工：本篇讲**基础语法打底**（程序骨架、变量、输入输出、分支循环、数组、字符串、结构体数组、指针入门）；**指针深入、`malloc`、链表**属于第二章的《C语言知识准备》，那里再展开。

0. 先看清"第一章要用哪些 C 知识"

绪论 / 登录案例里出现的东西	需要的 C 语法	出现在哪里
一个完整的 `.c` 程序	预处理指令、`main` 函数、头文件	所有程序
账号 `long id`、权限 `int pms`	基本数据类型、变量定义	`user` 结构体
密码 `char pwd[6]`	字符数组、字符串	登录校验
输入账号密码	`scanf` 与格式符 `%ld` `%d` `%s`	`main`
提示语、登录结果	`printf` 与转义字符	`main`
`strcmp(stupwd, users[i].pwd)`	字符串比较函数、`string.h`	校验逻辑
`if ... else` 判断权限	选择结构（含嵌套）	案例二
逐个比对找账号	`for` 循环 + 线性查找 + `break`	案例一、二
`if (i == n)` 判定失败	循环结束条件与边界判定	校验逻辑
`user users[50]`	结构体数组（顺序存储的雏形）	全局定义
`user *p = users;`	指针入门：指针访问数组	`init` 函数
数循环判断 O(1)/O(n)/O(n²)	循环嵌套	时间复杂度分析

知识地图：



```
结构体数组 user                数循环 → 时间复杂度
（顺序存储的第一次见面）        O(1) / O(n) / O(n²)
```

---

1. C 程序的骨架

1.1 完整结构

```c
#include <stdio.h>      // ① 预处理指令：把头文件"抄"进来
#include <string.h>     //    string.h 提供 strcmp、strcpy

#define N 50            // ② 宏定义：常量（后面写 N 就代表 50）

// ③ 全局变量定义（在所有函数之外，大家都能用）

int main() {            // ④ 主函数：程序唯一的入口
    // ⑤ 声明部分：定义变量
    // ⑥ 执行部分：干活
    return 0;           // ⑦ 返回值 0 = 正常结束
}
```

组成部分	作用	第一章代码中的例子
`#include <stdio.h>`	提供 `printf` / `scanf`	必写
`#include <string.h>`	提供 `strcmp` / `strcpy`	密码比较必写
`#define`	定义常量，改一处全变	可把 50 写成 `USERSIZE`
全局变量	定义在函数外，所有函数共享	`user users[50];`
`int main()`	程序入口，有且只有一个	所有程序
`return 0;`	告诉系统"正常结束"	所有程序

1.2 注释

```c
// 单行注释：解释这一行在干什么（C99 起支持，现在编译器都认）

/*
   多行注释：
   适合写在函数开头说明功能、参数、返回值
*/
```

> **好习惯：** 第一章实践部分的代码里，每个函数前都有注释说明用途——这就是"程序 = 数据结构 + 算法"里算法步骤的文字化，考试和答辩都加分。

---

2. 变量与数据类型

2.1 常用类型与格式符（必须背下来）

类型	含义	取值范围（典型）	`printf`/`scanf` 格式符
`int`	整型	±21 亿左右	`%d`
`long`	长整型	更大，学号/工号常用	`%ld`
`char`	字符型	单个字符	`%c`
`char []`	字符数组（字符串）	一串字符	`%s`
`float`	单精度浮点	6~7 位有效数字	`%f`
`double`	双精度浮点	15~16 位有效数字	`%lf`

```
long stuid;            // 学号用 long
char stupwd[6];        // 密码用字符数组
int  i, n;             // 计数器用 int
```

2.2 变量命名规则

- 只能用**字母、数字、下划线**，且不能以数字开头；
- 区分大小写（`id` 和 `Id` 是两个变量）；
- 不能用关键字（`int`、`return`、`if`…）；
- 见名知意：`stuid`、`stupwd`、`pms` 都比 `a`、`b` 强。

2.3 初始化是硬规矩

```
int sum;       // 未初始化，值是内存里的垃圾数
int sum = 0;   // 正确：定义的同时给初值
```

> 累加、计数、查找标志位这类变量，**定义时必须赋初值**，这是 C 语言最常见的低级错误来源。

2.4 `#define` 常量

```
#define USERSIZE 50
user users[USERSIZE];      // 以后想改成 100，只改 #define 一行
```

2.5 全局变量 vs 局部变量

第一章代码里 `user users[50];` 写在**所有函数之外**，所以 `init()` 和 `main()` 都能直接访问，不用传参：

```
user users[50];             // 全局变量：整个文件都能用，不初始化自动清零

void init(int n) {
    for (i = 0; i < n; i++) {
        scanf("%ld,%s", &users[i].id, users[i].pwd);    // 直接用，不用传进来
    }
}
```

对比	局部变量	全局变量
定义位置	函数内部	函数外部
使用范围	只在自己函数里	整个文件
初值	不初始化是**垃圾值**	不初始化自动为 0
第一章用法	`int i, n;`	`user users[50];`

---

3. 输入输出：`printf` 与 `scanf`

3.1 `printf` 输出

```
printf("请您输入账号和密码:");                    // 纯提示，不换行
printf("登录成功！\n");                            // \n 换行
printf("学号\t密码\n");                            // \t 制表符，用来对齐
printf("账号：%ld，权限：%d\n", tid, tpms);        // 格式符按顺序被变量替换
```

转义字符	作用
`\n`	换行
`\t`	水平制表（对齐列）
`\\`	输出一个反斜杠

> **规则：** 格式串里有几个 `%`，后面就要跟几个**类型匹配**的实参，顺序也不能错。

3.2 `scanf` 输入（三个易错点）

```
scanf("%d", &n);                        // ① 普通变量必须加 &
scanf("%ld,%s", &stuid, stupwd);        // ② 数组名本身是地址，不加 &
scanf("%ld,%s,%d", &tid, tpwd, &tpms); // ③ 逗号要原样输入：2001,123456,1
```

易错点	说明
忘记 `&`	`scanf("%d", n);` 会运行崩溃
字符数组多写 `&`	`scanf("%s", &pwd);` 类型不对（正确：`pwd`）

易错点	说明
格式串里的逗号	`"%ld,%s"` 要求输入 `2001,abc`，写成空格就读不进

> **实战提示：** 输入格式一定在提示语里写清楚示例，如 `请输入账号,密码(如 2001,123456):`。

4. 运算符与表达式

类别	运算符	说明	例子
算术	`+ - * / %`	`/` 整除（5/2=2），`%` 取余（5%2=1）	`n % 2 == 0` 判偶数
关系	`> >= < <= == !=`	结果是 1（真）或 0（假）	`i < n`
逻辑	`&&`、`&|&|`、`!`	与、或、非	`id == x && strcmp(...) == 0`
赋值	`= += -=`	`sum += i` 等价 `sum = sum + i`	累加
三目	`? :`	简化二选一	`pms == 1 ? "教学督导" : "普通教师"`

4.1 两个必踩的坑

```
if (tpms = 1) { }       // 错！= 是赋值，条件永远为真
if (tpms == 1) { }      // 对！== 才是比较
```

```
// && 的短路：左边为假，右边不再执行
if (p != NULL && p->id == 2001) { }   // 先判空再取成员，顺序不能反
```

4.2 三目运算符（第二章会大量用到）

```
printf("%s\n", pms == 1 ? "教学督导" : "普通教师");
// 等价于：
if (pms == 1) printf("教学督导\n");
else          printf("普通教师\n");
```

5. 选择结构：让程序会"判断"

5.1 四种形式

```
// ① 单分支
if (stuid == users[i].id) { ... }

// ② 双分支
if (i == n) printf("登录失败! \n");
else        printf("登录成功! \n");

// ③ 多分支（else if 链）
if (pms == 1)      printf("教学督导\n");
else if (pms == 2) printf("普通教师\n");
else               printf("管理员\n");

// ④ 嵌套（案例二：先校验账号密码，再判断权限）
if (tid == users[i].id && strcmp(tpwd, users[i].pwd) == 0) {
    if (tpms == 1) printf("教学督导\n");      // 内层判断
    else           printf("普通教师\n");
    break;
}
```

5.2 `switch` 写法（权限分支的另一种写法）

```
switch (pms) {
    case 1: printf("教学督导\n"); break;    // break 不能漏，否则会继续往下执行
    case 2: printf("普通教师\n"); break;
    default: printf("未知身份\n"); break;
}
```

> **注意：** `switch` 只能判断**整型/字符型**，不能判断字符串（密码比较必须用 `strcmp`）。

5.3 悬空 else

```
if (a > 0)
    if (b > 0) printf("A");
else printf("B");        // 这个 else 其实属于内层的 if!
```

建议： 哪怕只有一条语句也加上 `{}`，既避免悬空错误，也方便以后加代码。

---

6. 循环结构：让程序会"重复"

6.1 `for` 的三要素与执行顺序

```
for (i = 0; i < n; i++) {
    // 循环体
}
   ↑初始化      ↑条件        ↑更新
   只做一次     每轮前判断    每轮后执行
```

```
for (i = 0; i < n; i++)
      |          |          |
      ▼          ▼          ▼
   进入循环 → 条件为真？→ 执行循环体 → i++ → 回到条件判断
                 |
                 └─ 条件为假 → 退出循环（此时 i == n）
```

> **关键结论：** 循环**正常结束**（没被 `break` 打断）时，`i` 的值等于 `n`——这正是第一章用 `if (i == n)` 判断"没找到"的依据。

6.2 线性查找标准模板（第一章实践的核心）

```
for (i = 0; i < n; i++) {
    if (stuid == users[i].id && strcmp(stupwd, users[i].pwd) == 0) {
        printf("登录成功！\n");
        break;                  // 找到了，立刻跳出，不再白找
    }
}
if (i == n) {                   // 循环走完都没找到（i 没有被 break 打断）
    printf("账号或者密码错误！\n");
}
```

情况	循环结束时 `i` 的值	判定
第 k 个元素处 `break`	`i == k`	找到了
一直没找到	`i == n`	查找失败

> 这就是数据结构里**线性查找（Linear Search）**的 C 语言实现，时间复杂度 **O(n)**——用户在第 1 个还是第 n 个，差别很大。

6.3 `while` 与 `do-while`

```
while (i < n) { ...; i++; }         // 先判断，后执行（可能一次都不做）
do { ...; i++; } while (i < n);     // 先执行，后判断（至少做一次）
```

6.4 `break` 与 `continue`

```
break;      // 直接跳出整个循环
continue;   // 跳过本次循环剩下的语句，进入下一轮
```

6.5 双重循环 → 时间复杂度 $O(n^2)$

```
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        printf("%d ", matrix[i][j]);   // 一共执行 n × n 次
    }
}
```

---

7. 数组与字符串

7.1 一维数组

```
int score[50];              // 定义 50 个 int，下标 0 ~ 49
score[0] = 92;              // 访问第 1 个元素
int a[5] = {1, 2, 3, 4, 5}; // 定义时初始化
int b[5] = {0};             // 全部初始化为 0
```

- 下标**从 0 开始**，最后一个是 `长度-1`；
- C 语言**不检查越界**：`score[50]` 编译能通过，运行时可能崩溃或改坏别的变量；
- 数组名就是**首地址**（为指针做铺垫）。

7.2 字符数组与字符串

```
char pwd[6];    // 最多放 5 个有效字符，最后一位必须留给结束符 '\0'
```

操作	错误写法	正确写法	头文件
比较内容	`if (stupwd == users[i].pwd)`	`if (strcmp(stupwd, users[i].pwd) == 0)`	`string.h`
复制内容	`stupwd = users[i].pwd;`	`strcpy(stupwd, users[i].pwd);`	`string.h`
输入	`scanf("%s", &stupwd);`	`scanf("%s", stupwd);`	`stdio.h`

`strcmp` 的返回值（反直觉，务必记住）：

返回值	含义
`0`	两个字符串**相等**

返回值	含义
`< 0`	前者小于后者
`> 0`	前者大于后者

```
if (strcmp(tpwd, users[i].pwd) == 0) {   // 相等时返回 0，所以要写 == 0
    printf("密码正确\n");
}
```

> **安全提示（案例里的真实隐患）**：`char pwd[6]` 只能存 5 位密码，若用户输入 6 位以上，`scanf("%s", pwd)` 会**越界写入**，可能覆盖其他变量。实际开发要限制输入宽度，如 `scanf("%5s", pwd)`。

---

8. 结构体：把数据项打包（顺序存储的雏形）

8.1 定义用户类型

```
typedef struct {
    long id;          // 数据项：账号
    char pwd[6];      // 数据项：密码
    int  pms;         // 数据项：权限（案例二新增）
} user;               // 用 typedef 起个外号：以后 user 就是一种类型
```

对应绪论的概念：

绪论概念	代码中的体现
数据项	`id`、`pwd`、`pms`
数据元素	一个 `user` 变量（一位用户的完整信息）
数据	`users[50]` 这个整体（全部用户信息）

8.2 结构体数组 = 顺序存储

```
user users[50];               // 50 位用户，内存里连续存放
users[0].id = 2001;           // 先按下标取结构体，再用 . 取成员
strcpy(users[0].pwd, "123456");
users[0].pms = 1;
```

```
users[0]      users[1]      users[2]      ...
┌──────────┬──────────┬──────────┬──────────┐
| id  pwd  | id  pwd  | id  pwd  |          |   连续排队 = 顺序存储
└──────────┴──────────┴──────────┴──────────┘
    ↑ 这就是绪论讲的"顺序存储结构"在 C 语言里的样子
```

> **呼应理论**：绪论说"顺序存储 = 连续排队"。`users[50]` 就是最直白的顺序存储——知道下标就能一步到位找到人（随机访问 O(1)），这正是第二章顺序表的前身。

9. 指针入门：用指针访问数组

（深入部分见第二章准备篇，这里只讲第一章案例够用的程度）

9.1 数组名就是地址

```
user *p;
p = users;          // users 是数组首地址，赋给指针 p（不用加 &）
```

9.2 三种等价写法

写法	含义
`users[i]`	下标法
`p[i]`	指针当下标用（`p = users` 后与 `users[i]` 完全相同）
`*(p + i)`	地址偏移法：从首地址往后挪 i 个元素，再取值

```
p = users;
for (i = 0; i < n; i++) {
    scanf("%ld,%s", &p[i].id, p[i].pwd);    // 案例一 init() 中的写法
}
```

9.3 指针加 1 走多远？

```
p         p+1       p+2
  ▼         ▼         ▼
┌─────────┬─────────┬─────────┐
|user[0]|user[1]|user[2]|
└─────────┴─────────┴─────────┘
  +1 不是加 1 个字节，而是"往后挪一个 user 元素"
```

> **一句话：** 指针的 `+1` 由**类型**决定步长，`user*` 加 1 = 跳过一整个 `user` 结构体。

10. 函数：把一段功能"打包复用"

10.1 定义与调用

```
// 定义：返回值类型 函数名(参数列表) { 函数体 }
void init(int n) {                  // void = 不返回结果
    user *p;
    int i;
    p = users;
    printf("请管理员输入用户信息:\n");
    for (i = 0; i < n; i++) {
        scanf("%ld,%s", &p[i].id, p[i].pwd);
```

```
    }
}

// 调用
init(n);
```

10.2 参数传递的两条规则

参数类型	函数内修改能否影响外面	例子
普通变量（值传递）	**不能**（拿到的是副本）	`void f(int x)`
数组名 / 指针（地址传递）	**能**（拿到的是原件地址）	`void init(user arr[])`

```
void bad(int x) { x = 100; }      // 改的是副本
void good(int *x) { *x = 100; }   // 改的是原件

int a = 10;
bad(a);      // a 还是 10
good(&a);    // a 变成 100
```

> **为什么 `init(n)` 里改 `users` 外面能看到？** 因为 `users` 是**全局数组**（见 2.5 节）；如果改成局部变量，就必须把数组（或指针）当参数传进去——传数组本质是传地址。

---

11. 用 C 代码"数"出时间复杂度

绪论重点讲时间复杂度，落到代码上就是一句话：**看循环执行了多少次。**

代码形态	执行次数	时间复杂度
无循环，直接取值 `int x = arr[4];`	1 次	**O(1)**
单层循环 `for (i = 0; i < n; i++)`	n 次	**O(n)**
双层循环 `for ... for ...`	n × n 次	**O(n²)**
每次翻倍 `while (i <= n) i = i * 2;`	$\log_2 n$ 次	**O(log n)**
高斯公式 `sum = n * (n + 1) / 2;`	3 次运算	**O(1)**

```
// O(log n) 的典型样子：循环变量每次乘 2
int i = 1;
while (i <= n) {
    i = i * 2;      // 1,2,4,8,... 循环约 log₂n 次
}
```

分析三步走：

1. **找循环** —— 没有循环基本就是 O(1)；

2. **看循环变量怎么变** —— 每次 ±1 多半是 O(n)，每次 ×2 / ÷2 是 O(log n)；
3. **看嵌套** —— 内外层复杂度**相乘**；并列的代码段取**最大**的那段（加法法则）。

```
O(1)   —— 一步到位：直接取下标、高斯公式
O(log n) —— 每次砍一半：二分查找
O(n)   —— 从头到尾扫一遍：线性查找（登录校验）
O(n²)  —— 两层循环：冒泡排序、矩阵遍历
```

---

12. 高频错误对照表

错误写法	后果	正确写法
`if (tpms = 1)`	条件永远成立	`if (tpms == 1)`
`if (stupwd == users[i].pwd)`	比的是地址，永远不等	`strcmp(stupwd, users[i].pwd) == 0`
`scanf("%d", n);`	运行时崩溃	`scanf("%d", &n);`
`scanf("%s", &pwd);`	类型不匹配	`scanf("%s", pwd);`
忘写 `#include <string.h>`	`strcmp` 未声明	补上头文件
`for (i = 0; i <= n; i++)` 访问 `a[n]`	越界访问	`i < n`
`int sum;` 直接累加	初值随机，结果错误	`int sum = 0;`
`switch` 的 `case` 后忘 `break`	继续执行下一个 case	每个 case 加 `break`
`char pwd[6]` 输入 6 位以上密码	数组越界	`scanf("%5s", pwd)`
找到后不 `break`	白跑后面的循环	匹配成功立即 `break`
用 `i == n-1` 判定未找到	判定错误	`i == n`（break 时 i < n）
全局数组与局部数组搞混	函数间数据不共享	分清定义位置（见 2.5）

---

13. 本章小结

知识点	关键记忆
程序骨架	`#include` → 全局定义 → `main` → `return 0`
类型与格式符	`int`→`%d`、`long`→`%ld`、`char[]`→`%s`
输入	普通变量加 `&`，数组名不加；格式串里的逗号要原样输入
分支	`==` 才是比较；嵌套 `if` 实现权限二次判断

知识点	关键记忆
循环	`for` 三要素；`break` 找到就走；`i == n` 代表没找到
字符串	比较 `strcmp(...) == 0`、复制 `strcpy`、末尾有 `'\0'`
结构体数组	`user users[50]` —— 顺序存储的第一次见面
指针入门	`p = users;` 后 `p[i]` 等价于 `users[i]`
函数	值传递改不动外面，地址传递可以
时间复杂度	数循环：一层 O(n)、两层 O(n²)、翻倍 O(log n)、无循环 O(1)

14. 自测题（先做再对答案）

一、填空题

1. 定义长整型变量 `stuid`，输入时应写作 `scanf("_____", _____);`。
2. 判断两个字符串 `a`、`b` 是否相等，应写作 `if (_____(a, b) == _____)`。
3. 循环 `for (i = 0; i < n; i++)` 正常结束时，`i` 的值是 `_____`。
4. `char pwd[6]` 最多能存放 `_____` 个有效字符，因为要留一个位置给 `_____`。
5. `p = users;` 之后，`p[2]` 与 `_____` 完全等价。
6. 循环变量每次 `i = i * 2`，时间复杂度是 `O(_____)`。

二、程序阅读

写出下面程序的输出结果（输入：`3` 回车，再输入 `1001,aa` `1002,bb` `1003,cc`，再输入 `1002,bb`）：

```
#include <stdio.h>
#include <string.h>
typedef struct { long id; char pwd[6]; } user;
user users[10];
int main() {
    long sid; char spwd[6];
    int i, n;
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%ld,%s", &users[i].id, users[i].pwd);
    scanf("%ld,%s", &sid, spwd);
    for (i = 0; i < n; i++) {
        if (sid == users[i].id && strcmp(spwd, users[i].pwd) == 0) {
            printf("第%d个\n", i + 1);
            break;
        }
    }
    if (i == n) printf("查无此人\n");
    return 0;
}
```

三、找错

下面程序有 4 处错误，请指出并改正：

```
#include <stdio.h>
typedef struct { long id; char pwd[6]; } user;
user users[50];
int main() {
    long sid; char spwd[6];
    int i, n; flag = 0;
    scanf("%d", n);
    for (i = 0; i <= n; i++)
        scanf("%ld,%s", &users[i].id, &users[i].pwd);
    for (i = 0; i < n; i++)
        if (sid == users[i].id && spwd == users[i].pwd) flag = 1;
    if (flag = 1) printf("登录成功\n");
    return 0;
}
```

四、编程题

仿照登录案例，写一个"成绩查询"程序：结构体含 `long id`、`int score`，先输入 n 名学生的学号和成绩，再输入一个学号，查找并输出该生成绩；找不到则输出"查无此人"。（要求：找到后用 `break` 跳出，用 `i == n` 判定失败）

参考答案

一、填空题

1. `%ld` ；`&stuid`
2. `strcmp` ；`0`
3. `n`
4. `5` ；`'\0'`（字符串结束符）
5. `users[2]`
6. `log n`

二、程序阅读

```
第2个
```

（`1002,bb` 与 `users[1]` 匹配，`i` 为 1，输出 `i + 1 = 2` ；`break` 后 `i != n`，不输出"查无此人"）

三、找错

1. `int i, n; flag = 0;` → `flag` 未声明类型，应为 `int i, n, flag = 0;`（或单独写 `int flag = 0;`）。
2. `scanf("%d", n);` → 缺 `&`，应为 `scanf("%d", &n);`。
3. `for (i = 0; i <= n; i++)` → 越界（访问到 `users[n]`），应为 `i < n` ；且 `scanf("%ld,%s", &users[i].id, &users[i].pwd)` 中 `pwd` 是数组名，**不能加** `&`。
4. `if (flag = 1)` → 赋值写成比较，应为 `if (flag == 1)` ；另外密码比较 `spwd == users[i].pwd` 应改为 `strcmp(spwd, users[i].pwd) == 0`，并补上 `#include <string.h>`。

四、编程题（参考实现）

```c
#include <stdio.h>

typedef struct {
    long id;
    int  score;
} Stu;

Stu stus[50];

int main() {
    long sid;
    int i, n;

    printf("请输入学生人数: ");
    scanf("%d", &n);
    printf("请依次输入学号,成绩(如 1001,92):\n");
    for (i = 0; i < n; i++) {
        scanf("%ld,%d", &stus[i].id, &stus[i].score);
    }

    printf("请输入要查询的学号: ");
    scanf("%ld", &sid);

    for (i = 0; i < n; i++) {
        if (sid == stus[i].id) {
            printf("学号 %ld 的成绩是 %d\n", stus[i].id, stus[i].score);
            break;
        }
    }
    if (i == n) {
        printf("查无此人\n");
    }
    return 0;
}
```